UNIVERSIDAD TÉCNICA DE AMBATO

SISTEMA EMPRESARIAL BST

ESTUDIANTE:

Sivinta Lasluisa Cristian Fernando

CARRERA:

Ingeniería en Software

3ro – "B"

ASIGNATURA:

Estructura de Datos

FECHA:

27 – Abril – 2026

DOCENTE:

Ing. Jose Caisa, Mg.

AMBATO – ECUADOR

1. Objetivo General

Implementar un árbol binario de búsqueda en C++ para organizar empleados de una empresa.

2. Descripción del Problema

Una empresa requiere digitalizar su organigrama mediante una estructura de datos jerárquica. La solución propuesta implementa un Árbol Binario de Búsqueda donde cada nodo representa a un empleado. La clave de ordenamiento es el código numérico único de cada trabajador, lo que garantiza rapidez en las búsquedas y mantenimiento automático del orden.

2.1. Atributos de empleado

Cada nodo del árbol almacena un objeto de tipo Empleado con los siguientes atributos:

Atributo	Tipo de Dato	Descripcion
codigo	int	Identificador unico. Actua como clave de ordenamiento del BST.
nombre	string	Nombre completo del empleado.
cargo	string	Puesto o cargo dentro de la empresa.

3. Arquitectura del Sistema

El proyecto sigue una arquitectura modular basada en POO, separando las responsabilidades en tres archivos de cabecera/implementación y un programa principal:

Archivo	Clase / Modulo	Responsabilidad
Empleado.h / .cpp	Empleado	Define el modelo de datos: atributos, constructores, getters, setters y mostrarInfo().
ArbolBST.h / .cpp	ArbolBST	Encapsula la estructura del arbol: nodos, punteros raiz e implementacion de todos los algoritmos.
main.cpp	Programa principal	Interfaz de usuario (menu interactivo) que instancia el arbol e invoca sus metodos publicos.

4. Funcionalidades y Algoritmos

A continuación, se describe el comportamiento algorítmico de cada operación implementada en la clase ArbolBST.

4.1. Insertar Empleados

La inserción se realiza de forma recursiva mediante el método privado insertar(Nodo* nodo, Empleado emp). El algoritmo recorre el árbol comparando el código del nuevo empleado con el del nodo actual:

- Si el árbol esta vacío, se crea un nuevo Nodo y se asigna como raíz.

- Si el código del nuevo empleado es MENOR que el del nodo actual, se desciende por el subárbol IZQUIERDO.
- Si el código es MAYOR, se desciende por el subárbol DERECHO.
- Si el código ya existe, se imprime un mensaje de error (no se permiten duplicados).

Complejidad: O(log n) en caso promedio, O(n) en el peor caso (árbol degenerado).

```
// ArbolBST.cpp - Insercion recursiva
ArbolBST::Nodo* ArbolBST::insertar(Nodo* nodo, Empleado emp) {
    if (nodo == nullptr) return new Nodo(emp);

    if (emp.getCodigo() < nodo->empleado.getCodigo())
        nodo->izquierdo = insertar(nodo->izquierdo, emp);
    else if (emp.getCodigo() > nodo->empleado.getCodigo())
        nodo->derecho = insertar(nodo->derecho, emp);
    else
        cout << "Error: El codigo " << emp.getCodigo() << " ya existe.\n";

    return nodo;
}
```

4.2. Buscar un empleado por código

La búsqueda sigue la misma lógica de comparación que la inserción, aprovechando la propiedad de orden del BST para descartar mitades del árbol en cada paso:

- Si el nodo actual es nullptr, el empleado no existe en el árbol.
- Si el código buscado es igual al del nodo actual, se retorna ese nodo.
- Si es menor, la búsqueda continua en el subárbol izquierdo; si es mayor, en el derecho.

Complejidad: O(log n) en caso promedio, lo que representa una ventaja significativa frente a búsquedas lineales O(n) en listas.

4.3. Mostrar la Raiz del Árbol

El método mostrarRaiz() accede directamente al puntero raíz de la clase. Si el árbol no está vacío, muestra la información del nodo raíz. Esta operación es O(1). En el contexto del organigrama empresarial, el nodo raíz representa al nivel jerárquico más alto de la organización.

4.4. Recorridos de Profundidad

Los tres recorridos clásicos son implementados de forma recursiva. Su diferencia radica en el momento en que se procesa (visita) el nodo actual con respecto a sus subárboles:

Recorrido	Orden de Visita	Característica Principal
Inorden (LNR)	Izquierdo -> Nodo -> Derecho	Produce la lista de empleados ordenada de MENOR a MAYOR código.
Preorden (NLR)	Nodo -> Izquierdo -> Derecho	Útil para copiar o serializar la estructura del árbol. Siempre empieza por la raiz.
Postorden (LRN)	Izquierdo -> Derecho -> Nodo	Útil para liberar memoria (destrucción del árbol). Los hijos se procesan antes que el padre.

4.5. Calcular Altura del Árbol

El método privado altura(Nodo* nodo) calcula recursivamente la cantidad de niveles del árbol. La altura se define como el número de aristas en el camino más largo desde la raíz hasta una hoja:

- Caso base: si el nodo es nullptr, retorna 0.
- Caso recursivo: retorna 1 + max(altura(izquierdo), altura(derecho)).

La altura es un indicador directo de la eficiencia del árbol: un árbol balanceado de n nodos tiene altura O(log n), garantizando operaciones rápidas.

```
// ArbolBST.cpp - Calculo de altura

int ArbolBST::altura(Nodo* nodo) const {

    if (nodo == nullptr) return 0;

    return 1 + max(altura(nodo->izquierdo), altura(nodo->derecho));

}
```

4.6. Identificar y Mostrar Nodos Hoja

Un nodo hoja es aquel que no tiene hijos (ni izquierdo ni derecho son nullptr). El método mostrarHojas(Nodo* nodo) recorre el árbol y muestra los nodos que cumplen esta condición. En el organigrama empresarial, los nodos hoja representan a los empleados de nivel operativo sin subordinados.

```
// ArbolBST.cpp - Mostrar nodos hoja

void ArbolBST::mostrarHojas(Nodo* nodo) const {

    if (nodo != nullptr) {

        if (nodo->izquierdo == nullptr && nodo->derecho == nullptr)

            nodo->empleado.mostrarInfo();

        mostrarHojas(nodo->izquierdo);

        mostrarHojas(nodo->derecho);

    }

}
```

5. Pruebas de Escritorio

Se insertan 7 empleados en el siguiente orden de códigos: 50, 30, 70, 20, 40, 60, 80. A continuación se documenta el proceso de construcción del árbol y los resultados esperados de cada operacion.

5.1. Datos de Prueba

Código	Nombre	Cargo	Tipo de Nodo
50	Empresa UTA	Raiz	Raiz
30	Gerente Ventas	Nodo interno	Nodo Interno
70	Gerente Finanzas	Nodo interno	Nodo Interno
20	Emp 1	Hoja	Hoja
40	Emp 2	Hoja	Hoja
60	Emp 3	Hoja	Hoja
80	Emp 4	Hoja	Hoja

5.2. Estructura de Árbol Resultante

Tras insertar los 7 empleados, el árbol queda estructurado de la siguiente forma. Cada inserción se rige por la propiedad BST: códigos menores van a la izquierda y mayores a la derecha.

```
                [50] Empresa UTA (Raiz)
              /                    \
      [30] Gerente Ventas        [70] Gerente Finanzas
      /          \            /            \
 [20] Emp 1     [40] Emp 2  [60] Emp 3     [80] Emp 4
 (Hoja)         (Hoja)      (Hoja)         (Hoja)


Nivel 1: 50
Nivel 2: 30, 70
Nivel 3: 20, 40, 60, 80  <-- Nodos Hoja
```

Proceso de insercion paso a paso:

- Insertar 50 -> Arbol vacio, se convierte en la RAIZ.
- Insertar 30 -> 30 < 50, va al hijo IZQUIERDO de la raiz.
- Insertar 70 -> 70 > 50, va al hijo DERECHO de la raiz.
- Insertar 20 -> 20 < 50, baja a izquierda; 20 < 30, hijo IZQUIERDO de [30].
- Insertar 40 -> 40 < 50, baja a izquierda; 40 > 30, hijo DERECHO de [30].
- Insertar 60 -> 60 > 50, baja a derecha; 60 < 70, hijo IZQUIERDO de [70].
- Insertar 80 -> 80 > 50, baja a derecha; 80 > 70, hijo DERECHO de [70].

5.3. Resultados Esperados de los Recorridos

Recorrido Inorden (Izq -> Nodo -> Der) — Resultado ordenado ascendentemente:

[ID: 20] Emp 1 - Hoja -> [ID: 30] Gerente Ventas - Nodo interno -> [ID: 40] Emp 2 - Hoja

-> [ID: 50] Empresa UTA - Raiz -> [ID: 60] Emp 3 - Hoja -> [ID: 70] Gerente Finanzas - Nodo interno

-> [ID: 80] Emp 4 - Hoja

Recorrido Preorden (Nodo -> Izq -> Der) — Refleja la jerarquia de la raiz:

[ID: 50] Empresa UTA - Raiz -> [ID: 30] Gerente Ventas - Nodo interno -> [ID: 20] Emp 1 - Hoja

-> [ID: 40] Emp 2 - Hoja -> [ID: 70] Gerente Finanzas - Nodo interno -> [ID: 60] Emp 3 - Hoja

-> [ID: 80] Emp 4 - Hoja

Recorrido Postorden (Izq -> Der -> Nodo) — Los hijos antes que el padre:

[ID: 20] Emp 1 - Hoja -> [ID: 40] Emp 2 - Hoja -> [ID: 30] Gerente Ventas - Nodo interno

-> [ID: 60] Emp 3 - Hoja -> [ID: 80] Emp 4 - Hoja -> [ID: 70] Gerente Finanzas - Nodo interno

-> [ID: 50] Empresa UTA - Raiz

5.4. Resultados de Altura y Nodos Hoja

Operación	Resultado Esperado
Altura del Árbol	3 (Raiz[50] -> Gerente[30] -> Emp1[20])
Raiz del Árbol	[ID: 50] Empresa UTA - Raiz
Nodos Hoja (4 nodos)	[20] Emp 1 | [40] Emp 2 | [60] Emp 3 | [80] Emp 4
Buscar código 70	Encontrado: [ID: 70] Gerente Finanzas - Nodo interno
Buscar código 99	Empleado no encontrado.

6. Código Fuente

6.1. Empleado.h – Declaración de Clase Empleado

```
#ifndef EMPLEADO_H

#define EMPLEADO_H

#include <string>


class Empleado {

private:

    int codigo;

    std::string nombre;

    std::string cargo;
```

```
public:
    Empleado(int cod = 0, std::string nom = "", std::string car = "");

    // Getters
    int getCodigo() const;
    std::string getNombre() const;
    std::string getCargo() const;

    // Setters
    void setNombre(const std::string& nom);
    void setCargo(const std::string& car);

    // Metodos
    void mostrarInfo() const;
};
#endif
```

6.2. ArbolBST.h – Declaración de la clase ArbolBST

```
#ifndef ARBOLBST_H
#define ARBOLBST_H
#include "Empleado.h"

class ArbolBST {
private:
    class Nodo {
    public:
        Empleado empleado;
        Nodo* izquierdo;
        Nodo* derecho;
        Nodo(Empleado emp);
    };

    Nodo* raiz;

    // Metodos privados (recursivos internos)
```

```
    void vaciar(Nodo* nodo);
    Nodo* insertar(Nodo* nodo, Empleado emp);
    Nodo* buscar(Nodo* nodo, int codigo) const;
    Nodo* encontrarMinimo(Nodo* nodo) const;
    Nodo* eliminar(Nodo* nodo, int codigo);
    void inorden(Nodo* nodo) const;
    void preorden(Nodo* nodo) const;
    void postorden(Nodo* nodo) const;
    int altura(Nodo* nodo) const;
    void mostrarHojas(Nodo* nodo) const;

public:
    ArbolBST();
    ~ArbolBST();

    void insertarEmpleado(Empleado emp);
    void buscarEmpleado(int codigo) const;
    void eliminarEmpleado(int codigo);
    void actualizarEmpleado(int codigo);
    void mostrarRaiz() const;
    void mostrarInorden() const;
    void mostrarPreorden() const;
    void mostrarPostorden() const;
    void mostrarAltura() const;
    void mostrarNodosHoja() const;
};
#endif
```

6.3. Main.cpp – Menú interactivo

```
#include <iostream>
#include <string>
#include "Empleado.h"
#include "ArbolBST.h"
using namespace std;

int main() {
```

```
    ArbolBST arbol;
    int opcion, cod;

    do {
        cout << "\n===== SISTEMA EMPRESARIAL =====\n";
        cout << "1. Insertar   2. Buscar      3. Eliminar\n";
        cout << "4. Actualizar 5. Ver Raiz    6. Inorden\n";
        cout << "7. Preorden   8. Postorden   9. Altura\n";
        cout << "10. Ver Hojas 0. Salir\n";
        cout << "Opcion: ";

        if (!(cin >> opcion)) {
            cin.clear();
            cin.ignore(1000, '\n');
            continue;
        }

        switch (opcion) {
            case 1: { // Insertar
                int c; string n, car;
                cout << "Codigo: "; cin >> c; cin.ignore();
                cout << "Nombre: "; getline(cin, n);
                cout << "Cargo: ";  getline(cin, car);
                Empleado nuevoEmpleado(c, n, car);
                arbol.insertarEmpleado(nuevoEmpleado);
                break;
            }
            case 2: cout << "Codigo: "; cin >> cod;
arbol.buscarEmpleado(cod); break;
            case 3: cout << "Codigo a eliminar: "; cin >> cod;
arbol.eliminarEmpleado(cod); break;
            case 4: cout << "Codigo a actualizar: "; cin >> cod;
arbol.actualizarEmpleado(cod); break;
            case 5: arbol.mostrarRaiz(); break;
            case 6: arbol.mostrarInorden(); break;
            case 7: arbol.mostrarPreorden(); break;
            case 8: arbol.mostrarPostorden(); break;
```

```
            case 9: arbol.mostrarAltura(); break;

            case 10: arbol.mostrarNodosHoja(); break;

        }

    } while (opcion != 0);


    return 0;

}
```

7. Ejecución del código

7.1. Menú Principal

```
===== SISTEMA EMPRESARIAL =====
1. Insertar   2. Buscar      3. Eliminar
4. Actualizar 5. Ver Raiz    6. Inorden
7. Preorden   8. Postorden   9. Altura
10. Ver Hojas 0. Salir
Opcion: 
```

7.2. Inserción de Empleados

```
1. Insertar   2. Buscar      3. Eliminar
4. Actualizar 5. Ver Raiz    6. Inorden
7. Preorden   8. Postorden   9. Altura
10. Ver Hojas 0. Salir
Opcion: 1
Codigo: 50
Nombre: Empresa UTA
Cargo: Raiz

===== SISTEMA EMPRESARIAL =====
1. Insertar   2. Buscar      3. Eliminar
4. Actualizar 5. Ver Raiz    6. Inorden
7. Preorden   8. Postorden   9. Altura
10. Ver Hojas 0. Salir
Opcion: 1
Codigo: 30
Nombre: Gerente Ventas
Cargo: Nodo interno

===== SISTEMA EMPRESARIAL =====
1. Insertar   2. Buscar      3. Eliminar
4. Actualizar 5. Ver Raiz    6. Inorden
7. Preorden   8. Postorden   9. Altura
10. Ver Hojas 0. Salir
Opcion: 1
Codigo: 70
Nombre: Gerente Finanzas
Cargo: Nodo interno
```

7.3. Búsqueda de Empleado

```
Opcion: 2
Codigo: 70

Encontrado: [ID: 70] Gerente Finanzas - Nodo interno
```

7.4. Recorrido de Árbol

INORDEN

```
--- INORDEN ---
[ID: 0]  -
[ID: 20] Emp 1 - Hoja
[ID: 30] Gerente Ventas - Nodo interno
[ID: 40] Emp 2 - Hoja
[ID: 50] Empresa UTA - Raiz
[ID: 60] Emp 3 - oo
[ID: 70] Gerente Finanzas - Nodo interno
[ID: 80] Emp 4 - Hoja
```

PREORDEN

```
--- PREORDEN ---
[ID: 50] Empresa UTA - Raiz
[ID: 30] Gerente Ventas - Nodo interno
[ID: 20] Emp 1 - Hoja
[ID: 0]  -
[ID: 40] Emp 2 - Hoja
[ID: 70] Gerente Finanzas - Nodo interno
[ID: 60] Emp 3 - oo
[ID: 80] Emp 4 - Hoja
```

POSTORDEN

```
--- POSTORDEN ---
[ID: 0]  -
[ID: 20] Emp 1 - Hoja
[ID: 40] Emp 2 - Hoja
[ID: 30] Gerente Ventas - Nodo interno
[ID: 60] Emp 3 - oo
[ID: 80] Emp 4 - Hoja
[ID: 70] Gerente Finanzas - Nodo interno
[ID: 50] Empresa UTA - Raiz
```

7.5. Altura de Árbol y Nodos Hoja

ALTURA DEL ARBOL

```
Opcion: 9

Altura: 4
```

NODOS HOJA

```
--- HOJAS ---
[ID: 0]  -
[ID: 40] Emp 2 - Hoja
[ID: 60] Emp 3 - oo
[ID: 80] Emp 4 - Hoja
```

8. Análisis y Complejidad Algorítmica

Operacion	Operación	Peor Caso	Nota
Insertar	Insertar	O(n)	Peor caso: árbol degenerado

Buscar	Buscar	O(n)	Peor caso: árbol tipo lista
Eliminar	Eliminar	O(n)	Requiere búsqueda previa
Recorridos (In/Pre/Post)	Recorridos (In/Pre/Post)	O(n)	Siempre visita todos los nodos
Altura	Altura	O(n)	Recorre todos los nodos
Mostrar Raiz	Mostrar Raiz	O (1)	Acceso directo al puntero raíz

9. Conclusiones

Un Arbol Binario de Busqueda (BST) constituye una solucion elegante y eficiente para organizar informacion jerarquica en el contexto empresarial. A traves de la implementacion en C++ con Programacion Orientada a Objetos, se logro encapsular la complejidad de los algoritmos arboreos en una clase cohesiva (ArbolBST), manteniendo una interfaz publica clara y una separacion limpia de responsabilidades entre los modulos Empleado, ArbolBST y el programa principal.

La principal ventaja del BST radica en su complejidad algorítmica logarítmica O(log n) para las operaciones fundamentales de insercion, busqueda y eliminacion en casos promedio. Esto significa que, para un arbol de 1.000 empleados, una busqueda puede resolverse en aproximadamente 10 pasos, mientras que en una estructura lineal (lista o arreglo sin orden) se necesitarían hasta 1.000 comparaciones en el peor caso. Esta eficiencia se vuelve aun mas critica en sistemas empresariales reales donde el volumen de datos puede alcanzar decenas o cientos de miles de registros.

Los tres recorridos de profundidad (Inorden, Preorden y Postorden) ofrecen perspectivas complementarias de la misma estructura: el recorrido Inorden produce automaticamente un listado de empleados ordenado por codigo, el Preorden facilita la serializacion del arbol (util para copias de seguridad), y el Postorden es la base del destructor que libera la memoria de forma segura.

10. Bibliografía

[1] "Árboles de búsqueda binaria DSA." Accessed: Apr. 27, 2026. [Online]. Available: https://www.w3schools.com/dsa/dsa_data_binarysearchtrees.php

[2] "Binary Tree and Binary Search Tree (BST) | by Yogendra Pratap Singh (Tech Architect) | Medium." Accessed: Apr. 27, 2026. [Online]. Available: https://medium.com/@ys.yogendra22/binary-tree-and-binary-search-tree-bst-e6d7a4a20a4c

[3] "Binary Search Tree (BST) with Java Code and Examples | FavTutor." Accessed: Apr. 27, 2026. [Online]. Available: https://favtutor.com/blogs/binary-search-tree-java

11. Anexos

LINK GITHUB: https://github.com/Cristian-1402/SistemaEmpresarialBST.git